

AGEX Identity & Access Management Specification

S-05. AGEX IAM 공식 명세

문서 등급: Core Security & Platform Specification

상위 문서: AGEX Master Specification

연계 문서: Security Architecture / Multi-Tenant Architecture / Governance / API Specification / Administrator Guide / Developer Guide

상태: 공식 기준안

1. 문서 목적

본 문서는 AGEX에서 사용자, 서비스, Agent, Workload 및 외부 Identity를 식별하고, Tenant Membership·Role·Permission·Policy·Session·Credential·Privileged Access를 관리하기 위한 공식 **Identity & Access Management Specification**이다.

AGEX의 IAM은 단순 로그인 기능이 아니다.

본 명세는 다음 질문에 하나의 일관된 답을 제공해야 한다.

- 누가 요청을 했는가.
- 어떤 Tenant에 속하는가.
- 어떤 Role을 갖는가.
- 어떤 Permission을 갖는가.
- 어떤 Resource에 접근할 수 있는가.
- 어떤 조건에서 접근이 제한되는가.
- 어떤 Identity가 어떤 Credential을 사용할 수 있는가.
- Agent는 어떤 권한으로 실행되는가.
- Service와 Workload는 사람 계정과 어떻게 분리되는가.
- 관리자 권한은 어떻게 통제되는가.
- Tenant 간 접근은 어떻게 차단되는가.
- Session과 API Credential은 어떻게 만료·회수되는가.
- Authorization Decision은 어떻게 Audit되는가.

본 문서는 AGEX Zero Trust Security의 핵심 실행 계약이다.

2. 공식 정의

AGEX IAM은 모든 Human·Service·Workload·Agent Identity를 명시적으로 식별하고, Tenant Membership·Role·Permission·Policy·Context를 기반으로 모든 Resource 및 Action에 대한 접근을 최소권한 원칙으로 통제하는 AGEX의 공식 Identity and Authorization Control Plane이다.

3. IAM 핵심 원칙

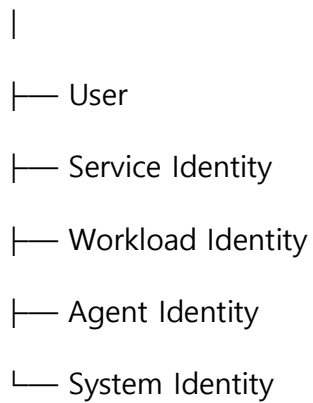
AGEX IAM은 다음 원칙을 따른다.

- Zero Trust
- Explicit Identity
- Tenant Context Mandatory
- Least Privilege
- Deny by Default
- No Shared Identity
- Separation of Authentication and Authorization
- Separation of User and Membership
- Separation of Role and Permission
- Separation of Permission and Policy
- Service Identity over Shared User Credential
- Short-Lived Credential Preferred
- Privileged Access Is Exceptional
- All Sensitive Access Is Auditable
- Cross-Tenant Default Deny
- Agent Cannot Self-Elevate

- Security Baseline Cannot Be Weakened by Tenant Policy
- Break-Glass Is Temporary
- Credential Rotation Is Mandatory Capability

4. IAM 전체 구조

Principal



Authentication



Tenant Membership



Role / Permission



Policy Evaluation





Authorization Decision



Runtime Enforcement

5. Principal

Principal은 AGEX에서 Action을 수행할 수 있는 식별 가능한 주체다.

공식 Principal Type:

USER

SERVICE

WORKLOAD

AGENT

SYSTEM

향후 Federation이 추가되면 External Principal을 확장할 수 있다.

6. Principal 공통 속성

최소:

principal_id

principal_type

display_name

state

created_at

updated_at

필요 시:

- tenant scope
- owner
- credential bindings
- identity provider
- labels

를 포함한다.

7. User Identity

User는 사람을 표현하는 Platform Identity다.

User Resource는 Tenant Role을 직접 포함하지 않는다.

구조:

User

├─ Identity Provider Mapping

└─ Memberships

8. User와 Membership 분리

가장 중요한 원칙 중 하나다.

User ≠ Tenant Membership

한 User는 여러 Tenant에서 서로 다른 Role을 가질 수 있다.

예:

User A

├─ Tenant X: Developer

├─ Tenant Y: Viewer

└─ Tenant Z: Administrator

9. User Resource

예시:

kind: User

metadata:

id: usr_xxx

scope_type: PLATFORM

specification:

display_name: Example User

primary_identity_provider: idp_xxx

status:

state: ACTIVE

10. User State

공식 기본 상태:

INVITED

ACTIVE

SUSPENDED

DISABLED

DELETED

Identity Provider 연동 방식에 따라 INVITED는 Membership 상태에서만 사용할 수도 있다.

11. Membership Resource

Membership은 User와 Tenant 간 소속 관계를 표현한다.

User



Membership



Tenant

12. Membership Schema

최소:

membership_id

user_id

tenant_id

roles

state

joined_at

expires_at

13. Membership State

공식:

INVITED

ACTIVE

SUSPENDED

REVOKED

EXPIRED

14. Membership Expiration

Temporary Access를 위해 Membership에 만료시간을 둘 수 있다.

만료된 Membership은 자동으로 Resource 접근 권한을 잃어야 한다.

15. Role

Role은 Permission 집합을 관리하기 위한 논리적 그룹이다.

Role

→ Permissions

Role 자체가 Resource Authorization의 최종 결정은 아니다.

16. Built-in Role

예:

- Viewer
- Developer
- Operator
- Tenant Administrator
- Security Administrator
- Governance Administrator

정확한 Built-in Role은 Edition에 따라 달라질 수 있다.

17. Custom Role

Tenant가 Custom Role을 만들 수 있다.

예:

Agent Operator

|— agent:read

|— agent:execute

|— execution:read

Wildcard 사용은 최소화한다.

18. Permission

Permission은 특정 Action에 대한 기본적 허용 가능성을 표현한다.

형식 예:

resource:action

예:

agent:create

agent:read

agent:execute

workflow:publish

plugin:install

audit:read

19. Permission과 Role

Role = Permission Collection

그러나 실제 실행 여부는 Policy와 Context까지 고려해야 한다.

20. Permission과 Policy 구분

공식:

Permission

≠

Policy

Permission:

이 Principal이 이 Action을 할 기본 권리가 있는가.

Policy:

현재 조건에서 실제로 허용되는가.

21. Authorization 최종 결정

최종 Authorization은 다음 교집합으로 계산한다.

Identity

$\cap$

Membership

$\cap$

Role

$\cap$

Permission

$\cap$

Tenant Scope

$\cap$

Resource Scope

$\cap$

Policy

$\cap$

Resource State

$\cap$

Runtime Context

22. Deny by Default

명시적으로 허용되지 않은 Action은 DENY다.

새 API나 Resource Type이 추가되었다고 기존 Broad Role에 자동 허용하지 않는다.

23. Resource Scope

Permission은 Resource Scope를 가져야 한다.

예:

Tenant-wide

Namespace

Specific Resource

Resource Type

가능한 최소 범위를 적용한다.

24. Wildcard Permission

예:

agent:*

같은 Wildcard는 Privileged Role에만 제한적으로 사용한다.

일반 Application Service에 사용하지 않는다.

25. Service Identity

Service Identity는 Backend Service나 외부 Application을 표현한다.

사람 계정을 자동화 Credential로 재사용하지 않는다.

26. Service Identity Schema

최소:

service_identity_id

name

tenant_id / platform_scope

owner

permissions

state

expires_at

last_used_at

27. Service Identity Owner

모든 Service Identity에는 Owner가 있어야 한다.

Owner가 사라지면 Review 대상이 된다.

28. Workload Identity

Workload Identity는 실행환경에서 자동 발급되는 단기 Identity를 표현한다.

예:

- Container
 - Pod
 - VM
 - Serverless Function
 - Runtime Worker
-

29. Workload Identity 장점

장기 API Key 없이도 Service 간 인증이 가능하다.

권장 구조:

Workload

→ Identity Provider

→ Short-lived Token

→ AGEX API

30. Agent Identity

Agent는 실행 Actor로서 별도 Identity Context를 가진다.

중요:

User Identity

≠

Agent Identity

≠

Execution Identity

31. Agent Authorization

Agent의 실제 Permission은 다음으로 계산한다.

Agent Permission

∩

Delegated Scope

∩

Tenant Policy

∩

Runtime Policy

User Permission 전체를 Agent가 자동 상속하지 않는다.

32. Agent Delegation

User가 Agent 실행을 요청했다고 해서 Agent에게 User의 모든 권한을 전달하지 않는다.

권장 구조:

User Request

→ Execution

→ Delegated Permission Scope

→ Agent Runtime

33. Delegated Permission

Execution 생성 시 필요한 Permission을 좁은 범위로 Delegation할 수 있다.

예:

User can manage all plugins

Agent execution delegated:

plugin:item.read only

34. Agent Self-Elevation 금지

Agent는 다음을 직접 수행할 수 없다.

- Role 변경
- Membership 생성
- Permission 추가
- Tenant 전환
- Admin 권한 획득
- Policy Override

35. System Identity

AGEX Platform 내부 시스템 작업에 사용한다.

예:

- Scheduler
- Reconciler
- Migration Worker
- Audit Processor

System Identity도 Audit 가능해야 한다.

36. Authentication

Authentication은 Principal이 누구인지 검증하는 단계다.

지원 가능 방식:

- OIDC
 - OAuth
 - SAML 연계
 - API Credential
 - Workload Identity
 - mTLS
 - Signed Service Credential
-

37. External Identity Provider

Enterprise 환경에서는 외부 Identity Provider와 연동할 수 있다.

예:

External IdP

→ Authentication

→ AGEX User Mapping

→ Membership

38. Identity Provider Resource

필드 예:

idp_id

type

issuer

client configuration

claim mapping

state

Secret은 Secret Reference로 관리한다.

39. Identity Mapping

외부 Claim을 내부 User/Membership에 Mapping한다.

예:

subject

email

group

organization

외부 Claim을 무검증 Role로 직접 변환하지 않는다.

40. SSO

Enterprise Tenant는 SSO를 사용할 수 있다.

SSO 설정은 Tenant Scope와 Platform Security Baseline을 모두 따라야 한다.

41. MFA

Privileged Role에는 MFA를 필수로 설정할 수 있다.

고위험 Action에서는 Step-Up Authentication을 요구할 수 있다.

42. Step-Up Authentication

예:

Normal Login

→ User session

Tenant Delete

→ Require fresh MFA

43. Session

Human User 로그인 후 Session을 생성할 수 있다.

Session에는 최소:

session_id

user_id

authentication_time

authentication_method

expires_at

revoked_at

을 관리한다.

44. Access Token

Access Token은 가능한 Short-Lived 방식으로 사용한다.

Long-lived Token에 Broad Permission을 부여하지 않는다.

45. Refresh Token

사용하는 경우:

- Secure storage
- Rotation
- Reuse Detection
- Revocation

을 적용한다.

46. Session Revocation

다음 상황에서 Session을 강제 종료할 수 있다.

- User Disabled
 - Admin Revocation
 - Credential Compromise
 - Security Incident
 - Membership Revoked
-

47. Session과 Membership

Session 생성 후 Membership이 Revoked되면 기존 Session도 해당 Tenant 접근을 즉시 잃어야 한다.

Session에 오래된 Role 정보를 영구 Cache하지 않는다.

48. API Credential

장기 Integration을 위해 API Credential을 사용할 수 있다.

하지만 가능하면 Workload Identity를 우선한다.

49. API Credential Schema

최소:

credential_id

principal_id

tenant_id

scopes

created_at

expires_at

last_used_at

revoked_at

Secret Value는 최초 생성 시에만 노출할 수 있다.

50. Credential Hashing

API Secret은 원문 저장을 피한다.

검증 가능한 Hash 또는 Secure Secret Store를 사용한다.

51. Credential Rotation

공식 Flow:

Create New

→ Deploy

→ Validate

→ Revoke Old

52. Credential Expiration

영구 Credential을 기본값으로 하지 않는다.

Expiry가 없는 Credential은 예외로 취급한다.

53. Credential Scope

Credential은 필요한 Permission만 가진다.

예:

agent:execute

execution:read

만 필요한 Service에 tenant:admin을 주지 않는다.

54. Privileged Access

다음 Permission은 Privileged로 분류할 수 있다.

- tenant:delete
 - user:admin
 - role:manage
 - policy:publish
 - secret:manage
 - model:block
 - plugin:block
 - audit:global_read
-

55. Privileged Role 관리

Privileged Role Assignment에는 다음을 적용할 수 있다.

- Approval
- Expiration
- JIT
- MFA
- Audit
- Periodic Review

56. Just-In-Time Access

고권한 권한은 필요한 시간 동안만 발급하는 것을 권장한다.

예:

Operator

→ requests elevated access

→ approval

→ 1 hour privileged role

→ auto expire

57. Privileged Access Expiration

Temporary Admin Role은 만료시간이 필수다.

만료 후 자동 제거한다.

58. Break-Glass Account

비상 접근용 Account.

조건:

- 제한된 수

- Strong MFA
 - Offline/secure credential storage
 - 사용 시 즉시 Alert
 - 강한 Audit
 - 사후 Review
-

59. Break-Glass 사용 금지

일상 Admin 작업에 사용하지 않는다.

60. Authorization Architecture

공식 구성:

PAP

PDP

PEP

PIP

PAP

Policy Administration Point

PDP

Policy Decision Point

PEP

Policy Enforcement Point

PIP

Policy Information Point

61. PAP

정책 생성·검토·Publish를 담당한다.

관리자 Console/API가 대표적이다.

62. PDP

Context를 받아 Authorization Decision을 반환한다.

입력:

Principal

Tenant

Action

Resource

Context

출력:

ALLOW

DENY

CONDITIONAL

63. PEP

실제 Action 전에 Decision을 강제하는 지점이다.

대표 PEP:

- API Gateway 이후 Domain Service
- Runtime
- Plugin Gateway
- Model Gateway
- Knowledge Retrieval Layer
- Secret Broker

64. PIP

Authorization에 필요한 Context를 제공한다.

예:

- Membership
- Role
- Resource Owner
- Data Classification
- Risk
- Region
- Time
- Device Trust

65. Authorization Decision Record

예:

```
{  
  "decision": "DENY",  
  "principal_id": "usr_123",  
  "tenant_id": "ten_123",  
  "action": "plugin.action.execute",  
  "resource_id": "pla_123",  
  "reason_code": "PERMISSION_MISSING",  
  "policy_ids": ["pol_1"]  
}
```

66. Conditional Decision

승인 등 추가 조건이 필요한 경우:

CONDITIONAL

과 Obligation을 반환할 수 있다.

예:

REQUIRE_APPROVAL

STEP_UP_AUTH

MASK_OUTPUT

67. Policy Hierarchy

공식 우선순위:

Platform Security Baseline

→ Environment Policy

→ Tenant Policy

→ Resource Policy

→ Execution Context

하위 Policy는 상위 DENY를 Override할 수 없다.

68. Tenant Authorization Boundary

Tenant Resource 접근 시 최소 다음을 확인한다.

Principal

→ Membership

→ Tenant Context

→ Resource Tenant

모두 일치해야 한다.

69. Cross-Tenant Default Deny

Tenant A Principal이 Tenant B Resource ID를 안다고 접근할 수 없어야 한다.

필수 Test:

Tenant A identity

+ Tenant B resource

→ DENY

70. Platform Scope Resource

Platform Administrator가 Platform Resource에 접근할 수 있다.

그러나 Platform Admin 권한이 Tenant Content 전체 읽기를 자동 허용하지 않는다.

71. Tenant Admin

Tenant Admin의 Scope는 해당 Tenant다.

Tenant Admin

≠

Platform Admin

72. Platform Admin Content Access

기본적으로 다음에 자유 접근하지 않는다.

- User Memory
- Private Knowledge Content
- Raw Prompt
- Tenant Secret

지원 목적에는 Support Session을 사용한다.

73. Support Session

Tenant Support Access를 위한 임시 Access Resource를 사용할 수 있다.

필드:

support_session_id

operator

tenant

reason

scope

expires_at

approved_by

74. Support Session 원칙

- Time Limited
 - Scope Limited
 - Audited
 - Reason Required
-

75. Impersonation

가능하면 사용자 완전 Impersonation보다 제한된 Support Context를 사용한다.

76. Resource Ownership

Resource에는 Owner를 지정할 수 있다.

Owner는 권한 정책에 사용될 수 있지만 Owner라는 이유만으로 무제한 Admin 권한이 생기지는 않는다.

77. Group / Team

향후 Group Resource를 지원할 수 있다.

Group은 다수 User를 묶어 Role/Ownership에 활용한다.

78. Group Membership

Group Membership과 Tenant Membership을 구분한다.

User가 Tenant에 속하지 않으면 Tenant Group에 포함될 수 없도록 한다.

79. Namespace Access

Tenant 내부 Namespace가 있다면 Role/Permission Scope를 Namespace까지 좁힐 수 있다.

80. Environment Access

Development/Staging/Production 권한을 분리할 수 있다.

예:

Developer

→ Development publish

Production Publisher

→ Production publish

81. Production Publish 권한

일반 Developer Role과 분리하는 것을 권장한다.

82. Secrets Access

Secret Permission을 다음처럼 세분화한다.

- secret:create
- secret:bind
- secret:rotate
- secret:revoke
- secret:metadata_read

secret:value_read는 가능하면 제공하지 않거나 극도로 제한한다.

83. Secret Binding

Agent/Plugin/Provider Resource는 Secret Value가 아니라 Reference만 가진다.

84. Model IAM

Model Gateway 요청도 Principal과 Tenant Context를 가진다.

확인:

- model:invoke
 - model profile permission
 - provider policy
 - data policy
-

85. Plugin IAM

Plugin Action 실행은 다음을 모두 확인한다.

Agent/User Permission

∩

Plugin Installation

∩

Action Permission

∩

Tenant Policy

86. Knowledge IAM

Knowledge Retrieval은 검색 이후가 아니라 Candidate 생성 전에 ACL을 적용한다.

87. Memory IAM

Memory Scope별 Access Rule을 둔다.

예:

- User Memory: 해당 User/허용 Agent
 - Agent Memory: 해당 Agent Scope
 - Tenant Memory: Tenant Policy
-

88. Workflow IAM

Workflow 실행 Permission과 Workflow 내부 Step Permission을 구분한다.

Workflow 실행이 허용되었다고 모든 내부 Plugin Action이 자동 허용되는 것은 아니다.

89. Agent-to-Agent IAM

Child Agent Permission:

Parent Delegation Scope

∩

Child Agent Permission

∩

Tenant Policy

Parent Agent의 Permission 전체를 상속하지 않는다.

90. Service-to-Service IAM

각 Service는 독립 Workload/Service Identity를 사용한다.

예:

Runtime Worker

Model Gateway

Plugin Runtime

Audit Processor

공유 Platform Token 하나를 사용하지 않는다.

91. mTLS

Private Deployment에서는 Service Identity 보강 수단으로 mTLS를 사용할 수 있다.

mTLS만으로 Authorization을 대체하지 않는다.

92. Token Audience

Service Token은 특정 Audience로 제한하는 것을 권장한다.

예:

aud = model-gateway

Model Gateway Token을 Admin API에 재사용하지 않는다.

93. Token Scope

Token에는 가능한 최소 Scope만 포함한다.

94. Token Replay 방지

가능하면:

- short expiry
- nonce/jti
- sender-constrained token
- mTLS binding

등을 고려한다.

95. Revocation

Revocation 대상:

- Session
 - API Credential
 - Service Credential
 - Membership
 - Privileged Role
 - Support Session
-

96. Revocation Propagation

중요 Revocation은 Cache TTL을 기다리지 않고 빠르게 전파되어야 한다.

특히:

- Admin Revocation
 - Tenant Suspension
 - Credential Compromise
-

97. Authorization Cache

Authorization Decision Cache를 사용할 수 있다.

Cache Key에는 최소:

principal

tenant

action

resource

policy_version

가 필요하다.

98. Cache Invalidation

다음 변경은 즉시 또는 빠른 Invalidation이 필요하다.

- Role change
 - Permission change
 - Membership revocation
 - Policy publish
 - Tenant suspension
-

99. Long-Running Execution과 IAM

Agent Execution이 장시간 지속되는 경우 시작 당시 Permission만 영원히 신뢰하지 않는다.

고위험 Action 시점에 재검증한다.

100. Execution Identity Snapshot

실행 시작 시 다음을 Snapshot으로 보관할 수 있다.

- initiating principal

- agent identity
- delegated scope
- policy references

단, Emergency Security Revocation은 Snapshot보다 우선할 수 있다.

101. Revalidation Trigger

다음 시점에는 재검증을 권장한다.

- Tool Side Effect
 - Approval 이후
 - Child Agent Delegation
 - Sensitive Data Export
 - Long Wait Resume
 - Secret Use
-

102. Approval과 IAM

Approval은 Permission을 생성하지 않는다.

Approval ≠ Permission

승인 후에도 실제 Permission을 다시 확인한다.

103. Entitlement와 IAM

Commercial Entitlement도 Permission을 생성하지 않는다.

Entitlement ≠ Authorization

104. Subscription 변경

Plan Upgrade로 기능이 활성화되어도 Role/Permission은 별도로 설정되어야 한다.

105. SCIM / Provisioning

Enterprise에서는 외부 Directory와 User/Membership 동기화를 지원할 수 있다.

106. SCIM 동기화 책임

가능한 항목:

- User create/update
 - Group sync
 - Membership provisioning
 - Deprovision
-

107. External Group → Role Mapping

직접 자동 Mapping은 신중하게 해야 한다.

권장:

External Group

→ AGEX Group

→ Role Binding

108. Deprovision

외부 Directory에서 사용자가 제거되면:

Disable User or Membership

→ Revoke Sessions

→ Revoke Credentials

→ Transfer Ownership

을 수행한다.

109. Ownership Transfer

사용자 Offboarding 전 Production Resource Owner를 확인한다.

Owner 없는 Active Resource를 방치하지 않는다.

110. Dormant Identity

오랫동안 사용되지 않는 다음을 탐지한다.

- User
- Service Identity
- API Credential
- Admin Role

Review 또는 Disable 후보로 관리한다.

111. Access Review

정기적으로:

- Privileged Role
- Custom Role
- Service Identity
- Support Access
- Break-Glass

를 검토한다.

112. Access Certification

Enterprise에서는 분기/반기 단위 Certification을 적용할 수 있다.

113. IAM Audit

반드시 Audit할 이벤트:

- Login success/failure
 - MFA change
 - Membership change
 - Role change
 - Permission change
 - API Credential create/revoke
 - Privileged access
 - Break-glass usage
 - Support session
 - Policy change
-

114. Audit 최소 필드

actor

target principal

tenant

action

old state

new state

reason

timestamp

request_id

Secret Value는 저장하지 않는다.

115. Authentication Failure Monitoring

다음 이상 징후를 탐지한다.

- 반복 로그인 실패
 - Credential stuffing
 - 비정상 Region
 - Admin Account 공격
-

116. Authorization Denial Monitoring

대량 Permission Denied는 다음을 의미할 수 있다.

- Application Bug
 - Misconfiguration
 - Attack
 - Privilege Escalation 시도
-

117. Privilege Escalation 탐지

예:

Agent tries admin endpoint

Service requests unrelated permission

User repeatedly accesses other tenant IDs

118. Tenant Context Mismatch

다음은 보안 이벤트 후보다.

Token Tenant A

Header Tenant B

즉시 DENY한다.

119. IDOR 방어

Resource ID만으로 Authorization을 결정하지 않는다.

모든 조회/수정에서 Tenant/Permission을 재검증한다.

120. Hidden Resource

보안상 Resource 존재 여부를 감출 필요가 있으면 403 대신 404를 사용할 수 있다.

정책은 일관되게 적용한다.

121. Authentication Error

대표 Error:

AUTHENTICATION_REQUIRED

INVALID_CREDENTIAL

SESSION_EXPIRED

TOKEN_REVOKED

MFA_REQUIRED

122. Authorization Error

대표:

PERMISSION_DENIED

TENANT_ACCESS_DENIED

POLICY_DENIED

PRIVILEGED_ACCESS_REQUIRED

DELEGATION_NOT_ALLOWED

123. API Header

대표:

Authorization: Bearer ...

X-AGEX-Tenant: ten_xxx

서버는 Header를 Token/Session과 교차 검증한다.

124. Public API와 Internal API

Internal Service API도 인증을 생략하지 않는다.

Workload Identity를 사용한다.

125. Anonymous Access

일부 Public Resource에 한해 Anonymous를 지원할 수 있다.

Default는 인증 필요하다.

126. Anonymous Principal

지원한다면 별도 Principal Type을 추가하기보다 Authentication Context의 unauthenticated 상태로 처리하는 것을 권장한다.

고위험 Resource에는 사용 금지.

127. API Key Query Parameter 금지

Credential을 URL Query String으로 전달하지 않는다.

Log/History에 노출될 위험이 있다.

128. Browser Credential

Browser에는 Long-lived Admin/Service Credential을 저장하지 않는다.

129. Backend-for-Frontend

고권한 작업:

Browser

→ Application Backend

→ AGEX API

구조를 사용한다.

130. CORS

CORS는 Browser Access 제어 도구이지 Authorization이 아니다.

131. Password Authentication

AGEX 자체 Password 시스템을 운영할 경우 별도 강한 보안 기준이 필요하다.

가능하면 Enterprise IdP/OIDC 연동을 우선 고려한다.

132. Password 저장

사용할 경우 강한 Password Hashing을 적용한다.

평문/복호화 가능한 Password 저장 금지.

133. Device Trust

고보안 환경에서는 Device Posture를 Policy Context로 사용할 수 있다.

MVP 필수는 아니다.

134. Conditional Access

조건 예:

- IP range
- region
- device trust
- time
- MFA state

Security Baseline 안에서 적용한다.

135. Network와 IAM

Network allowlist만으로 Identity/Authorization을 대체하지 않는다.

136. Admin Console IAM

메뉴 숨김은 UX일 뿐이다.

모든 Admin API는 Server-side Permission을 검증한다.

137. Console Session

Admin Console은 일반 User Session보다 짧은 Timeout을 사용할 수 있다.

138. Sensitive Action Confirmation

다음 Action은 Reauthentication을 요구할 수 있다.

- Tenant Delete
 - Global Plugin Block
 - Security Policy Change
 - Break-Glass Activation
-

139. IAM Resource Model

최소 공식 Resource:

User

Membership

Role

PermissionDefinition

RoleBinding

ServiceIdentity

Credential

Session

IdentityProvider

SupportSession

140. PermissionDefinition

Permission을 코드 문자열에만 흠뿌리지 않고 Registry로 관리할 수 있다.

필드:

permission_id

resource_type

action

risk

description

141. Role Resource

예:

kind: Role

specification:

permissions:

- agent:read
- agent:execute

Built-in Role도 동일 Contract를 따를 수 있다.

142. RoleBinding

Role을 Principal/Membership에 연결한다.

principal

role

scope

expires_at

143. Direct Permission Binding

필요 시 Role 없이 직접 Permission을 부여할 수 있지만 최소화한다.

운영 복잡성이 커지기 때문이다.

144. Permission Namespace

권장 형식:

domain:action

예:

agent:read

agent:create

agent:publish

agent:execute

workflow:execute

plugin:install

model:manage

145. Action Vocabulary

가능하면 CRUD 외 의미있는 Action을 사용한다.

예:

publish

activate

suspend

execute

approve

install

rotate

146. IAM Versioning

Role, Permission Model, Policy도 Version 관리가 필요하다.

특히 Published Policy는 Immutable Version을 사용한다.

147. Permission 변경

기존 Permission 의미를 바꾸는 것은 Breaking Security Change다.

새 Permission을 추가하는 방식이 안전하다.

148. Role 변경

Built-in Role에 새 고권한 Permission을 자동 추가하면 기존 User 권한이 조용히 확대될

수 있다.

따라서 보수적으로 관리한다.

149. Security Migration

Permission Model 변경 시:

old role

→ migration mapping

→ preview

→ apply

→ audit

를 권장한다.

150. IAM Testing

필수 테스트:

- Login
- Membership
- Role
- Permission
- Policy
- Token expiry
- Revocation
- Cross-Tenant
- Admin
- Service Identity
- Agent Delegation

151. Cross-Tenant Test

모든 Tenant-owned API에서 자동 수행한다.

152. Privilege Escalation Test

예:

Developer credential

→ tenant delete

→ DENY

153. Agent Escalation Test

L2 Agent

→ tries role update

→ DENY

154. Service Identity Test

Service가 허용되지 않은 Domain API 호출 시 DENY.

155. Revocation Test

Credential/Session Revocation 후 즉시 또는 정의된 최대 전파시간 안에 Access가 차단되어야 한다.

156. Long-running Revalidation Test

Execution 중 Membership Revoked 후 고위험 Action 시 DENY되는지 검증한다.

157. IAM Observability

Metric 예:

- auth_success
 - auth_failure
 - permission_denied
 - policy_denied
 - token_revoked
 - privileged_access
 - cross_tenant_denied
 - support_session_active
-

158. IAM Dashboard

관리자 Dashboard:

- Active Users
 - Privileged Users
 - Service Identities
 - Expiring Credentials
 - Denied Actions
 - Suspicious Cross-Tenant Attempts
 - Break-Glass Usage
-

159. IAM Alert

Critical Alert:

- Global Admin granted
- Break-Glass used

- Massive Cross-Tenant attempts
 - Credential compromise
 - Security policy bypass attempt
-

160. IAM API

대표 범주:

/users

/memberships

/roles

/role-bindings

/service-identities

/credentials

/sessions

/identity-providers

실제 URI는 API Specification에서 확정한다.

161. User API

대표:

GET /users/{id}

PATCH /users/{id}

User Tenant Role 수정은 Membership/RoleBinding API에서 수행한다.

162. Membership API

POST /memberships

PATCH /memberships/{id}

DELETE /memberships/{id}

163. Role API

GET /roles

POST /roles

PATCH /roles/{id}

Published/Built-in Role의 수정 정책은 별도로 둔다.

164. Credential API

POST /credentials

POST /credentials/{id}/rotate

POST /credentials/{id}/revoke

GET은 Secret Metadata만 반환하는 것을 권장한다.

165. Session API

관리자/User가 Active Sessions를 조회하고 Revoke할 수 있다.

166. SDK

SDK는 다음 기능을 제공할 수 있다.

agex.identity.users

agex.identity.memberships

agex.identity.roles

agex.identity.credentials

고위험 IAM API는 일반 Browser SDK에서 제한할 수 있다.

167. Developer Experience

Application 개발자가 IAM 내부 DB를 직접 다루지 않고 공식 API/SDK를 사용하도록 한다.

168. Multi-Tenant와 IAM

Tenant 생성 시 초기 Admin Membership을 명시적으로 생성한다.

Tenant 생성자가 자동 영구 Super Admin이 되는 구조로 고정하지 않는다.

169. Tenant Suspension

Tenant가 SUSPENDED되면 Membership이 ACTIVE여도 대부분 Tenant Action은 차단된다.

Membership Active

∩

Tenant Suspended

→ DENY

170. Billing Restriction

Billing Restricted와 Security Suspension을 구분한다.

특정 Read/Export 기능은 허용될 수 있다.

171. Identity Federation

향후 Cross-Organization Federation을 지원할 수 있다.

필요:

- Trust Domain
- External Principal Mapping
- Claims

- Tenant Acceptance
- Scope

MVP 필수는 아니다.

172. Cross-Tenant Federation

기본 IAM을 우회하는 기능으로 구현하지 않는다.

별도 Federation Authorization Contract가 필요하다.

173. Agent Federation

Remote Agent 호출 역시 Remote Identity와 Trust 계약이 필요하다.

174. IAM MVP 범위

초기 Production MVP에서는 최소 다음을 구현한다.

- User
- Tenant Membership
- Built-in Role
- Custom Role 기본
- Permission
- Role Binding
- OIDC/OAuth Authentication
- Session
- Service Identity
- API Credential
- Tenant Context
- Server-side Authorization

- Audit
 - Cross-Tenant Deny
 - Admin MFA 지원 구조
-

175. MVP 후순위

- Full SCIM
 - Device Trust
 - Advanced ABAC
 - JIT Admin Automation
 - Federation
 - Cross-Tenant Trust
 - Complex Conditional Access
 - Passwordless Advanced Flow
-

176. IAM 구현 우선순위

Phase A — Principal

- User
- Membership
- Service Identity

Phase B — Authentication

- OIDC
- Session
- API Credential

Phase C — Authorization

- Role

- Permission
- RoleBinding

Phase D — Tenant Enforcement

- Tenant Context
- Cross-Tenant Deny

Phase E — Policy

- PDP/PEP
- Conditional Decision

Phase F — Privileged Access

- Admin
- MFA
- Support Session
- Break-Glass

Phase G — Enterprise

- SCIM
- Federation
- JIT

177. IAM End-to-End Scenario — User

User authenticates

→ AGEX User resolved

→ Tenant selected

→ Membership validated

→ Role resolved

→ Permission evaluated

- Policy evaluated
 - API action allowed
 - Audit
-

178. IAM End-to-End Scenario — Agent

User starts Agent

- User authorized to execute Agent
 - Execution created
 - Delegated Agent Scope generated
 - Agent proposes Tool Action
 - Agent Permission checked
 - Tenant Policy checked
 - Approval if required
 - Tool executes
-

179. IAM End-to-End Scenario — Service

Backend Service

- Workload/Service Identity
 - Short-lived Token
 - Tenant Scope
 - agent:execute
 - AGEX API
-

180. IAM End-to-End Scenario — Revocation

Admin revokes Membership

- Authorization cache invalidated
 - Existing Session loses tenant access
 - Running Execution continues only within allowed snapshot
 - next sensitive action revalidation
 - DENY
-

181. Security Incident Scenario

Admin credential compromised

- Disable principal
 - Revoke sessions
 - Revoke credentials
 - Review audit
 - Rotate owned secrets
 - Transfer ownership
-

182. IAM Acceptance Criteria

AGEX IAM은 최소 다음 조건을 충족해야 한다.

1. User와 Membership이 분리된다.
2. 한 User가 여러 Tenant Membership을 가질 수 있다.
3. Tenant별 Role이 다를 수 있다.
4. Role과 Permission이 분리된다.
5. Permission과 Policy가 분리된다.
6. Deny by Default를 적용한다.
7. Service Identity를 지원한다.
8. Workload Identity 확장이 가능하다.

9. Agent Identity가 User Identity와 분리된다.
10. Agent가 User 전체 권한을 자동 상속하지 않는다.
11. Tenant Context를 모든 Tenant 요청에 적용한다.
12. Cross-Tenant 접근을 차단한다.
13. Session Revocation이 가능하다.
14. API Credential Rotation/Revocation이 가능하다.
15. Privileged Role을 식별할 수 있다.
16. MFA/Step-Up 구조를 지원할 수 있다.
17. Support Session을 제한적으로 제공할 수 있다.
18. Break-Glass Access가 Audit된다.
19. Authorization PDP/PEP 구조가 존재한다.
20. High-Risk Action에서 재검증할 수 있다.
21. Tenant Suspension이 IAM보다 우선 적용된다.
22. Entitlement와 Permission을 구분한다.
23. Approval과 Permission을 구분한다.
24. Role/Permission 변경이 Audit된다.
25. Credential Secret을 평문 재노출하지 않는다.

183. 금지사항

다음 IAM 구조를 허용하지 않는다.

- User Resource에 모든 Tenant Role 직접 저장
- 하나의 Global Role만으로 모든 Tenant 관리
- Shared Admin Account
- Service가 User Password를 사용
- Agent가 User Permission 전체 상속

- Agent가 자신의 Role 변경
- Client-side Tenant Filter로 Authorization 구현
- Resource ID만 알면 접근 가능
- Membership Revocation 후 기존 Session이 영구 접근
- Built-in Role에 고권한 Permission을 조용히 추가
- API Key를 URL Query에 전달
- Browser에 Global Admin Credential 저장
- Admin Console 메뉴 숨김을 보안으로 간주
- Support 목적으로 Platform Admin에게 상시 Tenant Content 접근 부여
- Break-Glass Account를 일상 운영에 사용
- Entitlement를 Permission으로 사용
- Approval을 Permission Grant로 사용
- External IdP Group을 검증 없이 Global Admin에 Mapping
- Security Policy DENY를 Tenant Role로 Override
- Credential Value를 Audit Log에 기록

184. IAM 공식 Resource 연결

본 명세는 다음 Resource를 사용한다.

User

Membership

Role

PermissionDefinition

RoleBinding

ServiceIdentity

WorkloadIdentity

Session

Credential

IdentityProvider

SupportSession

AuthorizationDecision

이 Resource는 Canonical Resource & Schema Specification에 Machine-readable 형태로 반영해야 한다.

185. Canonical Schema 추가 대상

다음 Schema를 추가한다.

/specs/schemas/identity/

├─ user.schema.json

├─ membership.schema.json

├─ role.schema.json

├─ permission.schema.json

├─ role-binding.schema.json

├─ service-identity.schema.json

├─ session.schema.json

├─ credential.schema.json

├─ identity-provider.schema.json

└─ support-session.schema.json

186. Permission Registry 추가

권장:

/specs/permissions/

- |— core.permissions.yaml
- |— agent.permissions.yaml
- |— workflow.permissions.yaml
- |— knowledge.permissions.yaml
- |— memory.permissions.yaml
- |— plugin.permissions.yaml
- |— model.permissions.yaml
- └— admin.permissions.yaml

Permission을 Source Code 여러 곳에서 임의 문자열로 정의하지 않는다.

187. 역할 정의 예

예:

role: Developer

permissions:

- agent:create
- agent:read
- agent:update
- agent:execute
- workflow:create
- workflow:read
- execution:read

Production Publish는 별도 Role에 둘 수 있다.

188. Privileged Role 예

role: TenantAdministrator

permissions:

- membership:manage
- role:manage
- quota:manage
- plugin:install
- model-profile:manage

Tenant Delete는 더 높은 별도 Permission으로 분리할 수 있다.

189. Authorization 핵심 명제

명제 1

인증되었다고 권한이 있는 것은 아니다.

명제 2

Role이 있다고 모든 Context에서 Action이 허용되는 것은 아니다.

명제 3

User가 가진 모든 권한을 Agent가 가질 필요는 없다.

명제 4

Tenant Context는 Authorization의 핵심 입력이다.

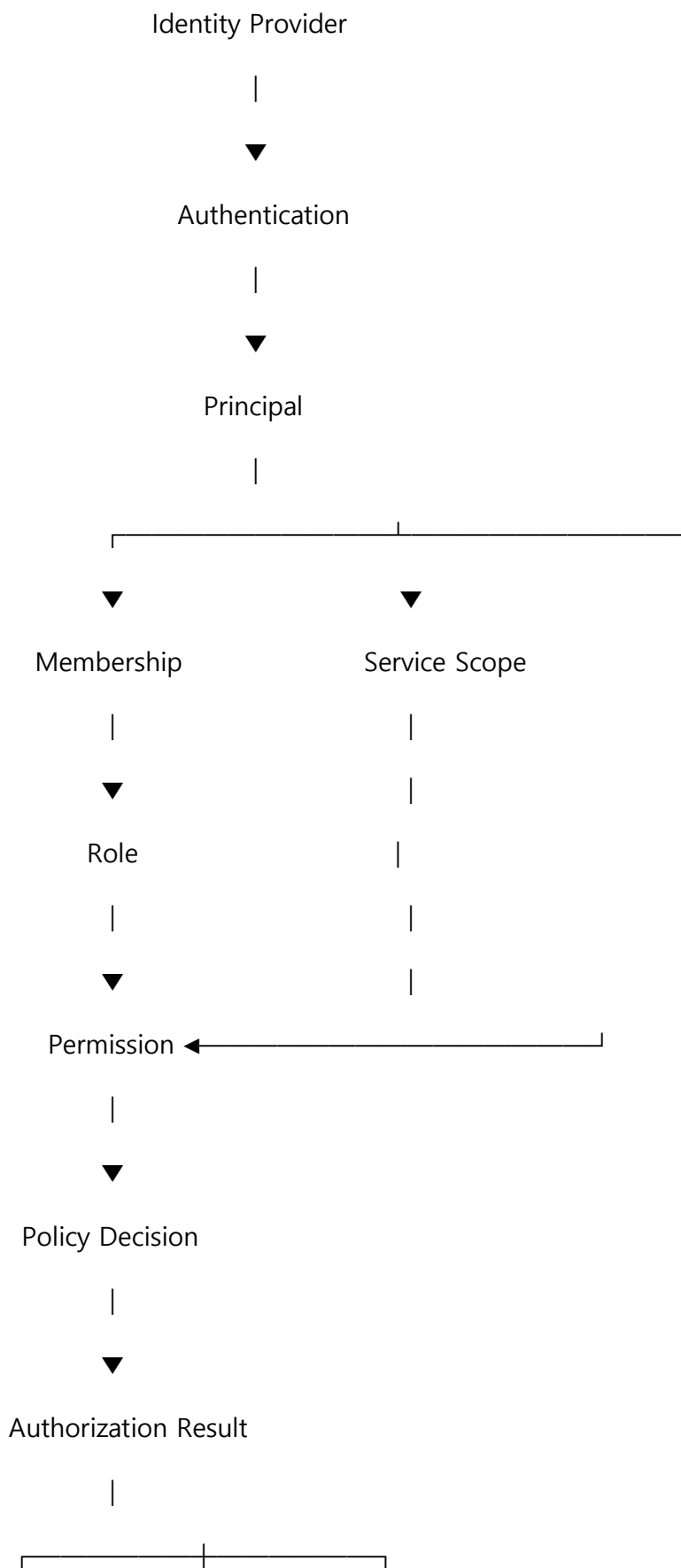
명제 5

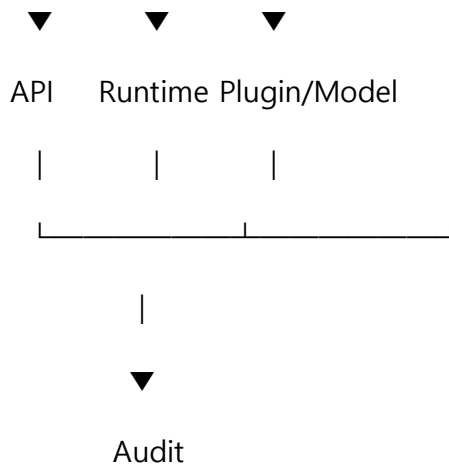
관리자 권한도 Zero Trust의 예외가 아니다.

명제 6

승인과 구매는 실행 권한이 아니다.

190. 최종 IAM Architecture





191. 최종 정의

AGEX Identity & Access Management는 사용자 로그인과 관리자 Role을 제공하는 단순 인증 시스템이 아니다.

AGEX IAM은 Human User, Service, Workload, Agent 및 System Actor를 각각 독립적인 Principal로 식별하고, Tenant Membership·Role·Permission·Policy 및 Runtime Context를 결합하여 모든 Resource Action의 실행 가능 여부를 판단하는 Zero Trust 기반 Access Control System이다.

User와 Tenant Membership은 분리하고 하나의 User가 여러 Tenant에서 서로 다른 Role을 가질 수 있도록 하며, Tenant-owned Resource 접근은 항상 Tenant Context와 Resource Tenant를 교차 검증한다.

Agent는 User 권한 전체를 자동 상속하지 않고 Execution에 명시적으로 Delegation된 Scope와 Agent 자체 Permission의 교집합 안에서만 행동한다.

Service와 Workload는 사람 계정을 공유하지 않고 전용 Identity와 가능한 Short-lived Credential을 사용한다.

Role은 Permission의 관리 단위일 뿐 실제 Authorization의 최종 기준이 아니며 Permission, Tenant, Policy, Resource State 및 Runtime Context를 모두 고려해 Decision을 내린다.

관리자 권한 또한 Zero Trust의 예외가 아니며 Privileged Access에는 MFA, JIT, Expiration, Audit 및 필요한 경우 Approval을 적용한다.

Approval, Commercial Entitlement 및 Resource Ownership은 각각 Authorization에 영향을 줄 수 있지만 자체적으로 Permission을 생성하지 않는다.

Membership, Session, Credential 또는 Privileged Role이 Revoked되면 캐시나 장기 Execution 때문에 권한이 무기한 남지 않도록 적절한 Revalidation과 Invalidation을 수행한다.

본 문서는 다음 문장으로 최종 정의한다.

AGEX IAM은 누가 시스템에 들어올 수 있는지를 관리하는 로그인 체계가 아니라, 모든 Human·Service·Agent 행위가 어떤 Tenant에서 어떤 Resource에 대해 어느 범위까지 허용되는지를 명시적 Identity·Permission·Policy 계약으로 통제하는 AI Operating System의 공식 Access Control Layer이다.